Notebook 02 : indexation du corpus et export pour le site

Ticket	#30, indexation du corpus
Notebook	`scripts/indexation/02_indexation.ipynb`
Pages de contrôle	trois pages HTML, dans `scripts/indexation/`
Auteur	Mahé BEGNIS
Équipe	Data Tigers, NLP 2, Master 2 Data & IA, FGES
Date	27 septembre 2026
Statut	exécuté intégralement ; index du site prêt ; évaluation en attente du jeu de questions

Objet. Ce document explique comment le notebook 02 transforme les passages du notebook 01 en index, quels contrôles garantissent que le site retrouvera les mêmes résultats, et comment le front doit lire l'export.

Tous les chiffres cités proviennent de l'exécution du 27 septembre 2026. La théorie utilisée (tokenisation, moyenne des tokens, préfixes, quantification, lecture des scores) est expliquée dans le support `docs/etudes/indexation/01_encodeur_e5_small.pdf`.

Ce document a été produit avec l'assistance d'outils d'intelligence artificielle. Il est relu par l'équipe dans la pull request qui l'introduit, et les usages seront consignés dans le journal prévu par le sujet.

Table des matières

1 En bref . . . 3
2 Place dans la chaîne . . . 3
3 Deux familles de fichiers pour un même modèle . . . 3
4 Les quatre contrôles . . . 4
4.1 Les requêtes de contrôle . . . 4
4.2 Contrôle 2 : l'effet de la quantification, simulé en Python . . . 4
4.3 Contrôle 3 : la fidélité, mesurée dans le navigateur . . . 4
4.4 Contrôle 4 : l'intégrité de l'export, vérifiée par le navigateur . . . 5
5 Encodage des découpages . . . 6
6 Spécification de l'export pour le site . . . 6
7 Index d'évaluation . . . 7
8 Exemples de recherche . . . 8
9 Questions probables en soutenance . . . 8

10 Reproduire 9
11 Limites et points ouverts 9

1 En bref

Question	Réponse
Qu'est-ce qui est indexé pour le site ?	les 4 240 passages du découpage M2, en vecteurs de 384 dimensions
Avec quel modèle ?	`intfloat/multilingual-e5-small` en fp32 pour l'index ; `Xenova/multilingual-e5-small` en q8 pour la question, dans le navigateur
Le navigateur calcule-t-il comme Python ?	en fp32, exactement ; en q8, presque : même premier résultat que le calcul exact pour 91,8 % des requêtes, et ce premier résultat reste dans le top 5 pour 99,9 %
L'export est-il fiable ?	oui : le navigateur vérifie les empreintes et classe les passages exactement comme Python, 24 requêtes sur 24
Que pèse l'export ?	12,3 Mio (index 6,21 Mio, passages 6,13 Mio), plus le modèle q8 de 112,8 Mio téléchargé depuis Hugging Face
Que reste-t-il à faire ?	mesurer la qualité de la recherche au notebook 03, avec le jeu de questions en préparation

2 Place dans la chaîne

Entrée	Sortie
`data/decoupage/passages/` : tables et manifeste du notebook 01	`data/indexation/site/` : index du site, métadonnées des passages, en-tête
modèles figés du Hub Hugging Face	`data/indexation/variantes/` : un index par découpage, pour le notebook 03
résultats des pages de contrôle, enregistrés dans `data/indexation/controle/`	`data/indexation/manifeste.json` : entrées, modèles, contrôles, fichiers produits

Le notebook vérifie d'abord que chaque table du notebook 01 a l'empreinte inscrite dans son manifeste, que chaque entrée porte le préfixe `passage:` et qu'aucune ne dépasse 512 tokens, recomptés indépendamment.

3 Deux familles de fichiers pour un même modèle

- **La référence** : `intfloat/multilingual-e5-small`, révision `614241f`, exécuté par PyTorch en fp32 via sentence-transformers, qui applique la moyenne des tokens puis la normalisation.
- **Les fichiers ONNX** de `Xenova/multilingual-e5-small`, révision `761b726`, ceux que Transformers.js charge dans le navigateur : `model.onnx` (fp32, 448,5 Mio) et `model_quantized.onnx` (q8, 112,8 Mio). Leurs empreintes sont vérifiées contre celles que publie le Hub.

Les fichiers ONNX ne renvoient qu'un vecteur par token. La fonction `encoder_onnx` du notebook reproduit ce que fait Transformers.js avec les options `{ pooling: "mean", normalize: true }` : moyenne des vecteurs des tokens réels, pondérée par le masque d'attention, puis division par la norme, en float32.

4 Les quatre contrôles

Chaque critère a été écrit avant la mesure, pour qu'aucun seuil ne soit choisi au vu des résultats.

Contrôle	Question	Critère fixé à l'avance	Résultat
1, parité	le modèle ONNX fp32 calcule-t-il comme la référence ?	tokenisation identique ; cosinus minimal de 0,99999	4 240 sur 4 240 ; cosinus minimal 0,99999976
2, quantification simulée	quel index servir avec une requête q8 ?	A, sauf si B fait significativement mieux (McNemar, 5 %)	A retenue
3, fidélité dans le navigateur	quelle précision servir ?	la meilleure variante 8 bits, q8 si l'écart n'est pas significatif	q8 retenue
4, intégrité	le site classe-t-il comme Python sur les fichiers exportés ?	empreintes conformes ; top 5 identique, ex aequo à 1e-6 près	24 requêtes sur 24

4.1 Les requêtes de contrôle

Le jeu de questions annotées n'existe pas encore. Pour comparer des précisions de calcul, il suffit de requêtes réalistes : les 822 titres de section formulés en questions (« Quelle est la durée de la période d'essai ? »), sur 1 301 titres distincts.

Ces titres figurent dans l'en-tête des passages M2 : la recherche les retrouve trop facilement. Ils mesurent l'accord entre deux calculs, jamais la qualité de la recherche, et ne servent jamais à comparer les découpages.

4.2 Contrôle 2 : l'effet de la quantification, simulé en Python

Dans le navigateur, la question est encodée par le modèle q8. Deux façons de construire l'index sont comparées au classement de référence, où tout est calculé en fp32.

Configuration	Index	Requête	Même premier résultat	Premier de référence dans le top 5	Rang maximal du premier de référence
A	PyTorch fp32	ONNX q8	96,5 %	100 %	2
B	ONNX q8	ONNX q8	93,2 %	100 %	4

Test exact de McNemar sur le premier résultat : A seule juste pour 38 requêtes, B seule pour 11, p = 0,00014. La configuration A est retenue : l'index est calculé une fois, au mieux, en fp32.

Index en entiers de 8 bits. Stocké en int8, l'index pèserait 1,55 Mio au lieu de 6,21 Mio, mais le premier résultat changerait pour 1,3 % des requêtes. Le gain est négligeable face aux 112,8 Mio du modèle : l'index reste en fp32.

4.3 Contrôle 3 : la fidélité, mesurée dans le navigateur

Un critère qui a échoué. La première version du notebook exigeait un cosinus d'au moins 0,9999 entre le vecteur q8 calculé par le navigateur et celui calculé en Python. Le cosinus minimal mesuré valait 0,9942 : le critère a échoué. Le diagnostic, rejoué et vérifié par le notebook sur 24 requêtes :

Étape isolée	Mesure	Conclusion
tokenisation de Transformers.js	identique à la nôtre pour 24 requêtes sur 24	juste
navigateur en fp32 contre référence	cosinus minimal 1,0	juste
moyenne recalculée à la main contre pipeline q8 du navigateur	cosinus minimal 1,0	juste
navigateur q8 contre q8 de Python	cosinus minimal 0,99419	écart

Tokenisation, moyenne et normalisation sont donc justes. C'est l'exécution du fichier q8 par ONNX Runtime Web qui diffère de celle d'ONNX Runtime en Python. Le critère reposait sur une hypothèse fausse, que les deux moteurs calculent pareil. Il est remplacé par deux contrôles distincts : la fidélité, mesurée dans le navigateur, et l'intégrité de l'export.

La mesure qui fait foi. La page `fidelite_navigateur.html` encode les 822 requêtes dans le navigateur, avec chaque précision du modèle. Le notebook compare les classements obtenus au classement de référence, sur l'index fp32.

Précision	Poids	Cosinus minimal avec la référence	Même premier résultat	Premier de référence dans le top 5	Rang maximal	Temps par requête
q8	112,8 Mio	0,99196	91,8 %	99,9 %	9	18,6 ms
int8	112,6 Mio	0,97326	85,8 %	99,3 %	23	18,1 ms
uint8	112,6 Mio	0,98262	90,8 %	99,9 %	8	18,2 ms
fp16	224,4 Mio	1,00000	99,9 %	100 %	2	16,7 ms
fp32	448,5 Mio	1,00000	100 %	100 %	1	16,6 ms

Le temps par requête couvre l'encodage seul, dans Chromium, en WASM. Deux sessions du navigateur ont donné des vecteurs identiques bit à bit pour les cinq précisions : seuls les temps varient d'une session à l'autre. La règle retient q8, la meilleure des variantes 8 bits. Avec les vrais vecteurs du navigateur, la configuration A reste la bonne : 91,8 % contre 91,4 % pour B, $p = 0{,}61$.

Lecture. La simulation Python du q8 sous-estimait l'erreur réelle du site : 96,5 % de premier résultat identique en Python, 91,8 % dans le navigateur. Pour un visiteur à qui l'on affiche 5 résultats, la différence est presque invisible : le premier résultat de référence y figure pour 821 requêtes sur 822.

Pourquoi pas fp16. fp16 reproduit le calcul exact (99,9 % de premier résultat identique), mais pour 111,6 Mio de plus au premier chargement. Le rapport J2 vise l'inverse : son risque R2 fixe une première visite d'environ 50 Mio, par réduction du vocabulaire. Une piste combine les deux : un modèle au vocabulaire réduit à 16 000 tokens, en fp16, pèserait environ 53 Mio (calcul à partir du nombre de paramètres, non mesuré). L'en-tête de l'index permet de changer de précision sans toucher au code du site.

4.4 Contrôle 4 : l'intégrité de l'export, vérifiée par le navigateur

La page `controle_navigateur.html` fait exactement ce que fera le site : elle lit l'en-tête, charge la matrice et le JSON, calcule leurs empreintes SHA-256 et les compare à celles de l'en-tête, charge le modèle et la précision indiqués, encode 24 requêtes et calcule leurs 5 premiers passages. Le notebook recalcule le classement avec le vecteur produit par le navigateur : ce contrôle isole l'intégrité de l'export de la précision du modèle.

Résultat : empreintes conformes, top 5 identique pour 24 requêtes sur 24, écart maximal entre les scores du navigateur et ceux de Python de $6 \cdot 10^{-8}$. La recherche exhaustive et le tri prennent 1,7 ms par question en médiane, 3,7 ms au plus.

5 Encodage des découpages

Tous les découpages sont encodés par la référence PyTorch fp32, puisque la configuration A est retenue. Contrôles sur chaque matrice : dimension 384, valeurs finies, norme égale à 1 à $1{,}8 \cdot 10^{-7}$ près, et deux entrées identiques donnent deux vecteurs identiques.

Index	Rôle	Passages	Entrées en double
`m2`	index du site	4 240	3
`m2_sans_entete`	effet de l'en-tête	4 240	57
`m2_256`	effet du budget de 256 tokens	7 009	6
`m0`	découpage de l'éditeur, référence	5 284	0
`m1`	fenêtre fixe	5 317	0
`m3`	découpage sémantique	4 229	2

Un passage encodé seul ou dans un lot donne le même vecteur, à $8{,}6 \cdot 10^{-8}$ près : le masque d'attention neutralise le remplissage. Deux exécutions sans cache donnent des matrices identiques octet pour octet (section 10).

6 Spécification de l'export pour le site

Trois fichiers, dans `data/indexation/site/`, à copier dans le dossier `public/` du site :

Fichier	Contenu	Taille	Compressé gzip
`index_m2.json`	l'en-tête : modèle, révision et précision à charger, préfixe et options de la requête, dimensions, empreintes	1 Kio	
`index_m2.f32`	la matrice, en float32 petit-boutiste, ligne par ligne : 4 240 × 384 nombres	6,21 Mio	5,70 Mio
`passages_m2.json`	un objet par passage, dans l'ordre des lignes de la matrice	6,13 Mio	1,27 Mio

Chaque objet de `passages_m2.json` porte `passage_id`, `titre_fiche`, `section_titre`, `chemin` (les intertitres), `url`, `url_citation` (le lien profond), `date_maj`, `texte` et `articles` (identifiants LEGIARTI, à compléter en `https://www.legifrance.gouv.fr/codes/article_lc/<LEGIARTI>`).

Le modèle n'est pas dans l'export. Transformers.js le télécharge depuis le Hub Hugging Face, à la révision indiquée par l'en-tête, puis le garde en cache dans le navigateur. Aucun des fichiers actuels du modèle ne pourrait être versionné : tous dépassent la limite de 100 Mio par fichier de GitHub. La réduction du vocabulaire prévue au J2 le ferait passer sous cette limite, selon un calcul qui reste à mesurer.

Règles de lecture, toutes lues dans l'en-tête :

1. lire l'en-tête d'abord ; refuser l'index si la taille de la matrice, le nombre de passages ou une empreinte ne correspond pas. C'est la garantie qu'un vecteur n'est jamais associé au mauvais passage ;
2. charger le modèle avec `dtype` explicite : par défaut, Transformers.js charge q8 en WASM mais fp32 ailleurs, WebGPU compris ;

3. encoder la question avec le préfixe `query: ` et les options `{ pooling: "mean", normalize: true }`. Par défaut, Transformers.js ne fait ni moyenne ni normalisation, et l'exemple de la carte du modèle renvoie un vecteur par token ;
4. calculer le produit scalaire avec chacun des 4 240 vecteurs : les vecteurs étant normalisés, c'est le cosinus. La recherche exhaustive suffit, sans base vectorielle : 1,7 ms par question en médiane, tri compris, mesurées dans le navigateur ;
5. trier par score décroissant et afficher le score, la section et le lien.

Le code minimal. Il a été exécuté tel quel dans le navigateur le 27 septembre 2026 : sur les 24 requêtes du contrôle 4, il renvoie exactement les mêmes 5 premiers passages et les mêmes scores que la page de contrôle.

```
import { pipeline } from "https://cdn.jsdelivr.net/npm/@huggingface/transformers@4.3.0";

const entete = await (await fetch("index_m2.json")).json();
const [octets, passages] = await Promise.all([
  fetch(entete.index.fichier).then((r) => r.arrayBuffer()),
  fetch(entete.metadonnees.fichier).then((r) => r.json()),
]);
const N = entete.index.nb_vecteurs, D = entete.index.dimension;
if (octets.byteLength !== N * D * 4 || passages.length !== N)
  throw new Error("index incohérent");
const index = new Float32Array(octets);

const extracteur = await pipeline("feature-extraction", entete.modele.depot,
  { revision: entete.modele.revision, dtype: entete.modele.dtype, device: "wasm" });

async function rechercher(question, k = 5) {
  const q = (await extracteur(entete.requete.prefixe + question,
    { pooling: entete.requete.pooling, normalize: entete.requete.normalize })).data;
  const scores = new Float32Array(N);
  for (let n = 0; n < N; n++) {
    let s = 0;
    for (let i = 0; i < D; i++) s += index[n * D + i] * q[i];
    scores[n] = s;
  }
  return Array.from(scores.keys()).sort((a, b) => scores[b] - scores[a]).slice(0, k)
    .map((n) => ({ score: scores[n], ...passages[n] }));
}
```

La vérification des empreintes par `crypto.subtle.digest("SHA-256", octets)` figure dans `controle_navigateur.html`, à reprendre telle quelle.

Lire les scores. Deux passages sans rapport ont un cosinus médian d'environ 0,83 : un score ne s'interprète qu'en relatif, comparé aux autres scores de la même question. Aucun seuil fixe n'a de sens ; un seuil d'affichage ou d'abstention se calibre sur le jeu de questions.

7 Index d'évaluation

Pour le notebook 03, chaque découpage est exporté dans `data/indexation/variantes/` : la matrice (`<nom>.f32`) et la liste ordonnée de ses identifiants de passage (`<nom>.ids.json`), décrites par `variantes.json`. Les métadonnées restent dans les tables du notebook 01, retrouvées par identifiant. Le notebook 03 n'aura rien

à encoder côté passages ; les questions du jeu d'évaluation devront, elles, être encodées dans le navigateur, avec la précision servie.

8 Exemples de recherche

Quatre requêtes, encodées en fp32 sur l'index du site. Ce sont des illustrations, pas une évaluation.

Requête	Premier résultat	Score
Quelles mentions sont interdites sur le bulletin de paie ?	Le bulletin de paie > Et les mentions interdites ?	0,9022
Mon employeur peut-il me licencier pendant un arrêt maladie ?	Les absences liées à la maladie ou à l'accident > Peut-il y avoir licenciement pour maladie ?	0,8846
Peut-on être viré quand on est en arrêt de travail ?	Les absences liées à la maladie ou à l'accident > Chapô	0,8764
Combien de temps peut durer la période d'essai d'un CDI ?	Le contrat à durée déterminée (CDD) > Quelle est la durée de la période d'essai ?	0,8927

L'échec, examiné. Pour la dernière requête, la fiche « La période d'essai », qui répond à la question, n'arrive qu'au rang 4, et son passage qui donne les durées maximales du CDI (article L. 1221-19) au rang 5, à 0,0089 du premier. En écrivant « contrat à durée indéterminée » au lieu de « CDI », la fiche passe au rang 1 et ce passage au rang 2. Une seule requête ne prouve rien, mais la piste est concrète : le jeu de questions doit contenir des sigles, et l'extension des sigles de la question est une amélioration à mesurer au notebook 03.

9 Questions probables en soutenance

Question	Réponse courte
Pourquoi deux modèles ?	c'est le même modèle, en deux formats : PyTorch pour calculer l'index au mieux, ONNX pour le navigateur ; le contrôle 1 prouve qu'ils calculent la même chose
Pourquoi l'index en fp32 et la question en q8 ?	l'index est calculé une fois, hors ligne ; le test apparié montre que cette configuration reproduit mieux le classement exact que l'index q8
Pourquoi le critère du navigateur a-t-il échoué ?	ONNX Runtime Web n'exécute pas le q8 exactement comme ONNX Runtime en Python ; le diagnostic l'a isolé étape par étape
Le q8 dégrade-t-il la recherche ?	il change le premier résultat pour 8,2 % des requêtes, mais le premier résultat exact reste dans le top 5 pour 99,9 % ; l'effet sur la qualité se mesurera au notebook 03
Comment sait-on que le site lira le bon index ?	l'en-tête porte les empreintes, que le navigateur recalcule ; ligne `i` de la matrice et passage `i` du JSON sont contrôlés
Pourquoi pas de base vectorielle ?	parcourir les 4 240 vecteurs prend 1,7 ms par question dans le navigateur, en médiane ; les index approchés d'une base vectorielle ne servent qu'à des corpus bien plus grands
Pourquoi un score de 0,88 n'est-il pas « bon » ?	les scores de ce modèle sont resserrés : deux passages sans rapport sont vers 0,83 ; seul l'ordre compte

10 Reproduire

Environnement. Depuis la racine du dépôt :

```
uv venv --seed --python 3.12 .venv
uv pip sync --python .venv/bin/python scripts/indexation/requirements-lock.txt
```

Ce fichier figé contient les dépendances du notebook 01, plus `onnxruntime` et ses deux dépendances. Le dossier `.venv` n'est pas dans le `.gitignore` : l'ajouter à `.git/info/exclude`. Le notebook 01 doit avoir été exécuté avant : le notebook 02 lit ses tables et contrôle leurs empreintes.

Exécution. Ouvrir le notebook dans VS Code avec le noyau `.venv`, puis tout exécuter. Le premier lancement télécharge les fichiers ONNX (448,5 et 112,8 Mio) et le modèle de référence (470 Mio) dans le cache Hugging Face, hors du dépôt. Les encodages sont mis en cache dans `data/indexation/cache/`, sous une clé qui dépend des textes, du modèle, de la taille de lot et des versions des bibliothèques. Sans cache, les encodages prennent environ 380 secondes au total sur CPU ; avec le cache, le notebook s'exécute en moins de 20 secondes.

Pages de contrôle. Les contrôles 3 et 4 demandent un vrai navigateur. Depuis la racine du dépôt :

1. exécuter le notebook une première fois : il écrit, dans `data/indexation/controle/`, les fichiers `attendu.json` et `requetes_fidelite.json` ;
2. lancer un serveur local : `python -m http.server 8765 --bind 127.0.0.1` ;
3. ouvrir dans Chrome `http://127.0.0.1:8765/scripts/indexation/controle_navigateur.html`, puis `diagnostic_navigateur.html` et `fidelite_navigateur.html` (cette dernière télécharge environ 1 Gio de modèles au premier lancement) ;
4. quand la page affiche « terminé », cliquer sur le lien de téléchargement et placer le fichier dans `data/indexation/controle/` ;
5. arrêter le serveur, puis réexécuter le notebook : les cellules des contrôles 3 et 4 lisent ces fichiers et vérifient leurs critères.

Tant qu'un fichier manque, la cellule correspondante l'indique au lieu d'échouer, et la précision servie reste q8 par défaut.

Déterminisme. Vérifié le 27 septembre 2026 : deux exécutions complètes sans cache ont produit 13 matrices sur 13 identiques octet pour octet, et des exports du site et des variantes aux mêmes empreintes. Dans le navigateur, deux sessions ont donné des vecteurs identiques bit à bit.

11 Limites et points ouverts

- **La qualité de la recherche n'est pas mesurée.** Les requêtes de contrôle mesurent l'accord entre deux calculs ; seule l'évaluation sur le jeu de questions annotées dira si la recherche est bonne.
- **Un seul navigateur mesuré.** La fidélité a été mesurée dans Chromium, en WASM, avec Transformers.js 4.3.0. Un autre navigateur, un autre moteur (WebGPU) ou une autre version pourrait donner un autre écart : le contrôle se rejoue avec la page de fidélité.
- **q8 contre fp16.** Décision d'équipe ouverte, voir le contrôle 3.
- **Le texte des passages alourdit le JSON.** 6,13 Mio bruts, 1,27 Mio compressés. Si le poids devient un problème, le texte complet peut être chargé à la demande.
- **Les sigles.** L'échec observé sur « CDI » suggère une faiblesse, à mesurer sur le jeu de questions.

- **Le cache d'encodage.** Sa clé couvre les textes, le modèle, la taille de lot et les versions des bibliothèques, mais pas le code de la fonction d'encodage : vider `data/indexation/cache/` après toute modification de ce code.